OSINT Suite

Application de veille et de capitalisation du renseignement

Élément	Détail
Plateformes	macOS 14+, iPadOS 17+, iOS 17+
Architecture	Monorepo Swift Package Manager, SwiftUI, MVVM
Données au repos	Archives chiffrées MLA (ANSSI) — chiffrement, compression, signatures
Cotations	OTAN / Admiralty Code : source A–F, information 1–6
IA	API Mistral via serveur proxy (clé jamais dans le client)
Réseau	Tout le trafic sortant via Tor (client Arti embarqué, proxy SOCKS5 local)
Synchronisation	Dépôt GitHub privé via Tor — deltas CRDT chiffrés (désactivable)
Cartographie	MapLibre — OSM, IGN (Etalab 2.0), EOX (CC BY 4.0)
Application compagnon	Gestionnaire d'accès : KEK/DEK, Secure Enclave, hors ligne

Ce guide décrit l'installation complète depuis le dépôt, la configuration (signature, trousseau, proxy Mistral, LaunchAgent) et l'utilisation au quotidien des quatre modules, du Gestionnaire d'accès et du collecteur.

1. Prérequis

Avant de commencer, vérifiez les éléments suivants :

- Mac sous macOS 14+ avec Xcode 15.4 ou plus récent (Swift 5.10) ;
- Compte développeur Apple payant (obligatoire pour le partage de trousseau entre les deux applications et le Secure Enclave) ;
- iPhone/iPad sous iOS 17+ / iPadOS 17+ pour les tests mobiles ;
- XcodeGen (génération du projet) :

```
brew install xcodegen
```

- Un serveur pour le proxy Mistral (option A validée) : petit VPS, Fly.io ou Render — ou votre Mac si le proxy reste local ;
- Abonnement Mistral Pro (clé API Mistral).

Répertoire des fichiers clés du dépôt :

Fichier / dossier	Rôle
project.yml	Définition XcodeGen du workspace (cibles, entitlements)
docs/XCODE.md	Guide de montage Xcode (détail de chaque réglage)
docs/SECURITE.md	Modèle de chiffrement KEK/DEK et flux de déverrouillage
App/OSINTSuite/	Application principale
App/AccessManager/	Gestionnaire d'accès (compagnon hors ligne)
App/LaunchAgent/	Plist du collecteur Mac hub (15 min)
Server/ProxyVapor/	Proxy Mistral (Vapor) — relais authentifié
Packages/	11 packages SPM (Domain, Persistence, Sync, Intelligence, Networking, Acces, 5 features + accueil)

2. Installation — mode pas-à-pas pour débutant

Cette partie part de zéro : aucun prérequis technique. Chaque action est numérotée, chaque commande est à copier-coller, et chaque étape se termine par « Ce que vous devez voir » pour vérifier que tout va bien avant de continuer. Comptez environ 1 à 2 heures la première fois (le plus long étant le téléchargement de Xcode).

2.1 Le vocabulaire indispensable (2 minutes)

Mot inconnu	En langage simple
Terminal	L'application Mac où l'on tape des commandes. Ouvrez-le avec Cmd+Espace, tapez « Terminal », Entrée.
Commande	Une ligne de texte à copier-coller dans le Terminal, terminée par la touche Entrée.
Xcode	L'atelier officiel d'Apple : c'est là que l'on fabrique et lance les applications.

Mot inconnu	En langage simple
XcodeGen	Un petit utilitaire qui fabrique automatiquement le fichier projet à partir de notre fiche de fabrication.
Team ID	L'identifiant de votre compte développeur Apple : 10 caractères (lettres et chiffres).
project.yml	La « fiche de fabrication » du projet. Vous n'y toucherez qu'une fois, pour y écrire votre Team ID.
Schéma (scheme)	Le menu déroulant en haut de Xcode qui choisit quelle application on lance.

2.2 Étape 1 — Installer Xcode

- **1.** Ouvrez l'App Store (icône bleue dans le Dock), cherchez **« Xcode »**, cliquez sur **Installer**. C'est gratuit. Le téléchargement est long (30 à 60 minutes selon votre connexion).
- **2.** À la fin, ouvrez Xcode une première fois : acceptez la licence (**Accept**), tapez le mot de passe de votre Mac, laissez-le finir ses installations.

Ce que vous devez voir : une fenêtre « Welcome to Xcode » avec les boutons Choose a project / Create a new project. Vous pouvez la fermer.

2.3 Découverte de Xcode — la fenêtre et les boutons (10 minutes)

Avant de continuer, prenez 10 minutes pour apprivoiser Xcode : les étapes suivantes l'utilisent sans explication. Rien à modifier ici — juste repérer les zones décrites ci-dessous.

Zone de Xcode	Où c'est, à quoi ça sert
Barre de titre (tout en haut)	Affiche le nom du projet ouvert. À droite : le menu déroulant du **schéma** (quelle app on fabrique) puis la **destination** (sur quelle machine on la lance : My Mac, iPhone…).
Bouton ■ (triangle)	À côté du schéma. C'est le bouton **Lancer** : il compile puis démarre l'application. Le carré ■ stoppe l'app lancée. Raccourci : **Cmd+R** pour lancer, **Cmd+.** pour arrêter.
Navigateur (colonne de gauche)	La liste des fichiers du projet. Les 3 petites icônes en haut de la colonne changent l'affichage : **dossier** = fichiers ; **rapport** (■+Cmd) = résultats des builds ; **warning** = problèmes.
Zone centrale (éditeur)	Affiche le fichier sélectionné — du code, ou les réglages quand on clique sur le projet en haut du navigateur.
Onglet Signing & Capabilities	Après avoir cliqué sur une cible (TARGETS) : c'est lé que l'on choisit son **Team** (signature automatique). C'est la seule zone de réglage utilisée par ce guide.
Bas de fenêtre	Pendant une compilation : une barre de progression et des messages. **Si une ligne rouge apparaît : le build a échoué** — cliquez dessus pour voir l'erreur en détail.

Vocabulaire Xcode utilisé dans la suite du guide :

Terme	Signification simple
Compiler (build)	Transformer le code en application. Xcode vérifie au passage toutes les erreurs. Raccourci : **Cmd+B** (compiler sans lancer).
Cible (target)	Une application à fabriquer. Ce projet en contient cinq : OSINTSuite-macOS, OSINTSuite-iOS, AccessManager-macOS, AccessManager-iOS, Collecteur.
Schéma (scheme)	Le menu déroulant qui dit « quelle cible on lance avec ■ ». On le change souvent : AccessManager d'abord, puis OSINTSuite.
Simulateur	Un iPhone/iPad virtuel affiché dans une fenêtre du Mac : gratuit, parfait pour découvrir sans compte payant (§2.16).
Build succeeded / failed	Le message final de la compilation : « succeeded » = tout est bon ; « failed » = il y a une erreur (souvent la signature).
Entitlements	Les autorisations spéciales demandées à Apple (partage de trousseau…). Déjà configurées dans ce projet : rien à faire.

*Trois réflexes à garder en tête : (1) lire la **zone en bas** de Xcode après chaque action ; (2) en cas d'erreur rouge, la **sélectionner** pour lire le message complet ; (3) en cas de doute : menu **Product > Clean Build Folder** (Maj+Cmd+K) puis relancer — cela résout la moitié des « fausses » erreurs.*

2.4 Étape 2 — Ouvrir le Terminal et installer les petits outils

XcodeGen ne s'installe pas par glisser-déposer : on passe par le Terminal. Rien de compliqué : trois copier-coller.

- **1.** Ouvrez le Terminal (Cmd+Espace, tapez « Terminal », Entrée). **Ce que vous devez voir :** une fenêtre avec une ligne qui se termine par **$** et un curseur qui clignote.
- **2.** Installez Homebrew (le « magasin d'outils » en ligne de commande) : copiez la ligne ci-dessous, collez-la dans le Terminal, validez avec Entrée.

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Pendant l'installation, le Terminal peut demander le mot de passe du Mac (rien ne s'affiche quand vous tapez : c'est normal, tapez-le puis Entrée). À la toute fin, il peut afficher deux petites commandes « Next steps » : copiez-collez-les aussi, une par une.

- **3.** Installez XcodeGen :

```
brew install xcodegen
```

Ce que vous devez voir : le Terminal défile avec des pourcentages, puis la ligne avec le **$** revient sans message d'erreur.

2.5 Étape 3 — Récupérer le code de l'application

Deux méthodes : la méthode A se fait uniquement à la souris — commencez par elle.

Méthode A (recommandée pour débuter) — à la souris :

- **1.** Ouvrez la page du dépôt dans Safari : github.com, puis recherchez reSqur-dygcus-wumvu8 / V2 ;
- **2.** Cliquez sur le gros bouton **vert « Code »**, puis **« Download ZIP »** ;
- **3.** Dans Téléchargements, double-cliquez sur **V2-main.zip** pour le décompresser ;

- **4.** Glissez le dossier obtenu dans **Documents** et renommez-le simplement **V2** (facile à retrouver ensuite).

Méthode B — dans le Terminal (une seule commande) :

```
cd ~/Documents &amp;&amp; git clone https://github.com/reSqur-dygcus-wumvu8/V2.git
```

Ce que vous devez voir : un dossier nommé **V2** dans Documents, contenant notamment project.yml, Packages et App.

2.6 Étape 4 — Trouver votre Team ID (gratuit, 5 minutes)

Bon à savoir : un Team ID existe même avec un compte Apple **gratuit** (on l'appelle alors « Personal Team »). Vous n'avez pas besoin du programme payant (99 $/an) pour l'obtenir — seulement de votre identifiant Apple habituel (celui de l'App Store).

Méthode A (la plus simple) — dans Xcode :

- **1.** Ouvrez **Xcode** ;
- **2.** Menu **Xcode > Settings…** (ou le raccourci **Cmd+,**) ;
- **3.** Cliquez sur l'onglet **Accounts** en haut ;
- **4.** Si la liste est vide : bouton **+** en bas → **Apple ID** → connectez-vous avec votre identifiant Apple habituel (identifiant + mot de passe, code de vérification sur votre iPhone) ;
- **5.** Cliquez sur votre compte dans la liste : en bas à droite, la zone **Team** affiche une ligne du type « Votre Nom (Personal Team) » suivie du code à 10 caractères — par exemple **AB12CD34EF**. C'est votre Team ID ;
- **6.** Notez-le sur papier ou dans l'app Notes — vous en aurez besoin à l'étape suivante.

Méthode B — sur le site d'Apple :

- **1.** Ouvrez Safari sur **developer.apple.com/account** ;
- **2.** Connectez-vous avec votre identifiant Apple (même gratuit) ;
- **3.** La page affiche **Membership details** : la ligne **Team ID** contient vos 10 caractères.

Je n'ai AUCUN compte et je ne veux rien créer : dans ce cas laissez le Team ID de côté, passez directement au §2.16 (simulateur). Le simulateur ne demande ni compte ni signature : c'est la voie 100 % gratuite et 100 % immédiate. Vous complèterez le Team ID plus tard si vous voulez aller sur appareil réel.

Le compte développeur payant (99 $/an) reste indispensable pour installer les applications sur un iPhone/iPad réel et pour le partage de trousseau entre les deux apps. Avec un compte gratuit, le Team ID suffit pour la signature Mac et le simulateur ; en revanche Apple refuse de délivrer les capacités « App Groups / Keychain Sharing » aux comptes gratuits : si la signature Mac échoue avec ces capacités, utilisez le simulateur (§2.16) ou passez au compte payant.

2.7 Étape 5 — Mettre votre Team ID dans le fichier project.yml

- **1.** Dans le dossier **V2** (Documents), double-cliquez sur le fichier **project.yml** : il s'ouvre dans TextEdit ;
- **2.** IMPORTANT : dans TextEdit, menu **Format > Convertir en texte simple** (sinon TextEdit ajoute du gras et le fichier devient illisible pour l'outil) ;
- **3.** Menu **Édition > Rechercher > Remplacer** : dans « Rechercher », tapez __**TEAM_ID**__ (deux tirets du 8, TEAM_ID, deux tirets du 8) ; dans « Remplacer par », collez votre Team ID ; cliquez **Tout** : 3 remplacements sont faits ;

- **4. Fichier > Enregistrer**, fermez TextEdit.

Ce que vous devez voir : dans project.yml, les lignes « fr.osintsuite.shared » sont précédées de votre Team ID, et il ne reste aucun __**TEAM_ID**__ dans le fichier.

2.8 Étape 6 — Fabriquer le projet Xcode

- **1.** Retournez dans le **Terminal** ;
- **2.** Tapez **cd** (cd suivi d'un espace, sans valider), puis glissez-déposez le dossier **V2** depuis le Finder jusque dans la fenêtre du Terminal : le chemin s'écrit tout seul. Validez avec Entrée ;
- **3.** Tapez la commande :

```
xcodegen generate
```

Ce que vous devez voir : quelques lignes se terminant par « Created project at .../OSINTSuite.xcodeproj » et un nouveau fichier **OSINTSuite.xcodeproj** est apparu dans le dossier V2.

2.9 Étape 7 — Ouvrir le projet et activer la signature

- **1.** Double-cliquez sur **OSINTSuite.xcodeproj** : Xcode s'ouvre et télécharge les composants (la première fois, patientez) ;
- **2.** Dans la colonne de gauche, cliquez sur l'icône en forme de **dossier** en haut, puis sélectionnez la ligne **OSINTSuite-macOS** (section TARGETS) ;
- **3.** Au centre, onglet **Signing & Capabilities** : cochez **Automatically manage signing** si ce n'est pas déjà fait, puis ouvrez le menu **Team** et choisissez votre nom (celui de votre compte développeur) ;
- **4.** Recommencez pour les autres cibles : **AccessManager-macOS**, puis **OSINTSuite-iOS** et **AccessManager-iOS**.

Ce que vous devez voir : à chaque cible, aucun message rouge — au pire un triangle jaune sur la cible Collecteur (outil macOS sans impact sur les apps).

2.10 Étape 8 — Lancer les applications sur le Mac

- **1.** En haut de Xcode, cliquez sur le menu déroulant à côté de « OSINTSuite » (le **schéma**) : choisissez **AccessManager-macOS** ;
- **2.** Cliquez sur le bouton **triangle** ■ ou pressez **Cmd+R**. La première fois, macOS peut signaler « développeur non identifié » : faites clic droit sur l'icône de l'app > **Ouvrir** ;
- **3.** Autorisez l'appareil courant (bouton « Autoriser cet appareil… ») et générez la phrase de récupération — notez-la sur papier ;
- **4.** Changez le schéma pour **OSINTSuite-macOS**, relancez avec ■.

Ce que vous devez voir : l'écran « Base verrouillée » de l'OSINT Suite. Le bouton « Demander le déverrouillage » bascule vers le Gestionnaire d'accès : la chaîne complète fonctionne.

Variante — construire et lancer sans ouvrir Xcode (Terminal seul)

Vous ne voulez jamais ouvrir la fenêtre de Xcode ? C'est possible : Xcode doit être **installé** sur le Mac (le compilateur et l'outil xcodebuild en font partie), mais vous pilotez tout depuis le Terminal. Quatre commandes suffisent, à taper après les étapes 2.4 à 2.8 (outils installés, code récupéré, Team ID dans project.yml, projet généré) :

- **1.** Placez-vous dans le dossier du projet :

```
cd ~/Documents/V2
```

• **2.** Fabriquez l'application macOS (sans signature — suffisant pour lancer sur votre propre Mac) :

```
xcodebuild -scheme OSINTSuite-macOS -configuration Debug \ -derivedDataPath build
CODE_SIGNING_ALLOWED=NO build
```

• **3. Ce que vous devez voir :** la dernière ligne affiche **** BUILD SUCCEEDED **** — patientez quelques minutes la première fois (téléchargement des composants).

• **4.** Lancez l'application :

```
open build/Build/Products/Debug/OSINTSuite.app
```

• **5.** La première fois, macOS peut signaler « développeur non identifié » : clic droit sur l'icône de l'app > **Ouvrir**.

Limites de cette variante : (1) l'app n'est pas signée, donc le partage de trousseau avec le Gestionnaire d'accès peut être refusé par macOS — pour la chaîne complète de déverrouillage, la signature via Xcode (§2.9) ou un compte payant reste nécessaire ; (2) cette variante construit la cible macOS ; pour iOS, le simulateur reste la voie gratuite (§2.16) ; (3) si xcodebuild affiche « xcode-select: error », collez d'abord : sudo xcode-select -s /Applications/Xcode.app/Contents/Developer

2.11 Ça ne marche pas ? Les 6 pannes classiques

me	Solution	
nd: brew »	Homebrew n'a pas fini son installation : relancez l'étape 2.3 et lisez les « Next steps » affichés à la fin (deux commandes à copier-coller).	
r » en	Xcode n'est pas désigné comme outil par défaut : ouvrez Xcode une fois (étape 2.2), ou collez : sudo xcode-select -s /Applications/Xcode.app/Contents/Developer	
e menu Team	Xcode n'a pas encore récupéré votre compte : Xcode > Settings > Accounts > sélectionnez votre compte > Download Manual Profiles, puis rouvrez le menu Team.	
sintsuite.app were found »	Xcode ne trouve pas de profil de signature pour le compte utilisé (compte gratuit, équipe non choisie, ou App ID déjà pris par un autre développeur)	1) Signing & Capabilities : vérifier que le menu **T** chaque cible ; 2) compte gratuit : passer par le si 3) bundle ID déjà pris : remplacer fr.osintsuite.ap fr.osintsuite.app.votrenom (idem .acces) ; 4) dern Product > Clean Build Folder puis redémarrer Xc
ou bundle	Un autre développeur utilise fr.osintsuite.app : dans Signing & Capabilities, remplacez le Bundle Identifier par fr.osintsuite.app.votrenom (faites-le aussi pour .acces).	
gras ou de	TextEdit l'a converti en texte riche : refaites l'étape 2.6 en commençant par Format > Convertir en texte simple AVANT de modifier.	
ectory » avec	Vous avez glissé le ZIP au lieu du dossier : décompressez V2-main.zip d'abord, glissez le dossier V2 obtenu.	

Si aucune de ces solutions ne suffit : ouvrez un ticket sur le dépôt GitHub en copiant le message d'erreur complet (et son numéro de ligne), la correction sera apportée rapidement.

2.12 Installation sur iPhone et iPad

Deux méthodes selon votre usage :

Méthode A — Développement direct (câble, compte gratuit ou payant)

- Connectez l'iPhone/iPad au Mac par câble ;
- Dans Xcode, sélectionnez votre appareil comme destination (il doit être enregistré : Signing & Capabilities > Team) ;
- Schemes **OSINTSuite-iOS** et **AccessManager-iOS** : Cmd+R pour installer et lancer chaque application sur l'appareil ;
- Sur l'appareil : Réglages > Général > VPN et gestion des appareils > votre profil développeur > **Approuver** ;
- Note : la signature « free » expire tous les 7 jours (réinstaller via Xcode) ; avec un compte payant, 1 an et jusqu'à 100 appareils.

Méthode B — TestFlight (compte développeur payant, recommandé)

```
xcodebuild -scheme OSINTSuite-iOS -configuration Release build \ -archivePath
build/OSINTSuite.xcarchive -destination 'generic/platform=iOS' xcodebuild
-exportArchive -archivePath build/OSINTSuite.xcarchive \ -exportOptionsPlist
ExportOptions.plist # puis : App Store Connect > TestFlight > téléverser, inviter les
testeurs
```

- Les testateurs installent via l'app TestFlight depuis l'invitation ;
- Renouvelez l'opération pour le **AccessManager** (les deux apps doivent être installées sur l'appareil pour le déverrouillage).

2.13 Spécificités iPhone/iPad

Sujet	Comportement sur iPhone/iPad
Exécution en arrière-plan	iOS n'autorise pas la veille continue : l'appareil est « consomme uniquement » — rafraîchissement à l'ouverture + BGTaskScheduler quand le système le permet
Rôle de l'appareil	Réglages > Gérer > Rôle : laissez « consomme uniquement » ; le Mac collecte (LaunchAgent 15 min) et les résultats arrivent par synchronisation
Déverrouillage	Le Gestionnaire d'accès s'authentifie par Face ID / Touch ID ou code, puis délivre la clé de session à durée limitée
Notifications	Pull via Tor aux rafraîchissements + notifications locales (les APNs contournent Tor : refusés par défaut)
Carte hors ligne	Préchargez la zone AVANT de partir hors connexion (Paramètres ou vue Carte) — le cache tuiles se consulte ensuite sans réseau
Fenêtres multiples	iPadOS : glissez l'app en Split View / Slide Over depuis le Dock (identique macOS)

2.14 (Optionnel) Vérifier que tout est bon : les tests automatiques

Si vous souhaitez vérifier le code avant de lancer (optionnel) :

```
for p in Packages/PackageDomain Packages/PackageMLA Packages/PackageTor \
Packages/PackagePersistence Packages/PackageNetworking \
Packages/PackageIntelligence Packages/PackageSync \ Packages/PackageAcces; do (cd $p
&amp;&amp; swift test); done
```

Ce que vous devez voir : à la fin de chaque paquet, une ligne « TEST SUCCEEDED » en vert. Un « TEST FAILED » rouge : copiez le message pour le ticket.

2.15 Option 2 — Installer sans Xcode sur le Mac (GitHub + TestFlight)

Cette variante compile l'application sur les machines de GitHub (Xcode y est déjà installé) et la publie sur TestFlight : vous n'installez rien sur votre Mac. En contrepartie, la préparation des clés de signature est plus délicate (une bonne heure la première fois) et le compte développeur payant (99 $/an) est obligatoire. À réserver à ceux qui ne peuvent pas installer Xcode (Mac trop ancien ou trop rempli). Pour tous les autres, la méthode classique (étapes 2.2 à 2.9) reste plus simple.

A. Ce qu'il vous faut avant de commencer

- Un compte **Apple Developer payant** (99 $/an) — obligatoire pour TestFlight ;
- Un navigateur web (tout se fait sur des sites web et dans GitHub, jamais dans Xcode) ;
- Votre **Team ID** (étape 2.5 du guide) — notez-le ;
- 15 minutes de patience pour les téléchargements automatiques.

B. Créer les clés de signature sur le site Apple (une seule fois)

B1. Créer un certificat de distribution :

- 1. Ouvrez l'app **Trousseau d'accès** du Mac (Cmd+Espace, tapez « Trousseau ») ; menu **Trousseau d'accès > Assistant de certificat > Demander un certificat à une autorité de certification** ;
- 2. Entrez votre adresse e-mail, un nom commun quelconque, cochez « Enregistré sur le disque », **Continuer** : un fichier CertificateSigningRequest.certSigningRequest est créé ;
- 3. Sur le site développeur Apple : **Certificates, Identifiers & Profiles > Certificates > +** > choisissez **Apple Distribution** > téléversez le fichier .certSigningRequest ;
- 4. **Téléchargez** le certificat obtenu (fichier .cer) ;
- 5. Double-cliquez sur le .cer : il s'installe dans le trousseau ;
- 6. Dans Trousseau d'accès, catégorie **My Certificates** (Mes certificats) : cliquez la petite flèche devant votre certificat pour afficher la clé privée ; sélectionnez **les deux lignes** (certificat + clé) ; clic droit > **Exporter 2 éléments** ; nommez-le **distribution**, format .p12, choisissez un **mot de passe simple** (ex. osint2024) — notez-le, c'est le P12_PASSWORD ;
- 7. Fichier obtenu : **distribution.p12** — ne le partagez jamais (il signe vos applications).

B2. Créer le profil de provisionnement :

- 1. Sur le site développeur Apple : **Profiles > +** > **App Store Connect** > continuez ;
- 2. Choisissez l'App ID **fr.osintsuite.app** (créez-le au besoin : Identifiers > + > App > description « OSINT Suite », Bundle ID explicite fr.osintsuite.app) ;
- 3. Sélectionnez votre certificat de distribution > **Générer** > **téléchargez** le fichier .mobileprovision ;
- 4. Notez le nom exact du profil affiché (ex. « OSINT Suite App Store ») ;
- 5. IMPORTANT : recommencez pour l'App ID **fr.osintsuite.acces** (le Gestionnaire d'accès doit être publié séparément).

B3. Créer une clé API App Store Connect :

- 1. Sur **appstoreconnect.apple.com** : icône de vos initiales en haut > **App Store Connect > Gérer** > onglet **Intégrations > Clés d'équipe** > **+ Générez une clé API** ;
- 2. Nom : « github-actions » ; accès : **Admin** (ou au minimum App Manager) ; **Générer** ;
- 3. La page affiche TROIS informations à noter immédiatement : **ID de la clé** (Key ID), **ID d'émetteur** (Issuer ID), et le bouton **Télécharger la clé API** (fichier AuthKey_XXXXXXXX.p8 — **impossible à retélécharger ensuite !**) ;

B4. Créer l'app dans App Store Connect :

- Sur appstoreconnect.apple.com : **Mes apps > + Nouvelle app** ; nom « OSINT Suite », langue français, Bundle ID fr.osintsuite.app, SKU « osintsuite » ; répétez pour « Gestionnaire d'accès » avec fr.osintsuite.acces et le SKU « osintacces ».

C. Déposer les secrets dans GitHub

Sur github.com : votre dépôt > **Settings > Secrets and variables > Actions > New repository secret**. Créez SEPT secrets — pour les fichiers, il faut d'abord les convertir en texte base64 :

- 1. Ouvrez le Terminal ; glissez-y le dossier contenant vos fichiers (distribution.p12, .mobileprovision, AuthKey.p8) ;
- 2. Convertissez chacun en base64 (remplacez les noms par vos fichiers) :

```
base64 -i distribution.p12 -o cert.txt\nbase64 -i distribution.mobileprovision -o profile.txt\nbase64 -i AuthKey_ABC123.p8 -o key.txt\n# puis ouvrez chaque .txt et copiez tout son contenu
```

Nom du secret GitHub	Contenu à coller
APPLE_TEAM_ID	Votre Team ID (10 caractères)
BUILD_CERTIFICATE_BASE64	Tout le contenu de cert.txt
P12_PASSWORD	Le mot de passe choisi en B1.6
KEYCHAIN_PASSWORD	Un mot de passe quelconque (ex. osint-keychain)
PROVISIONING_PROFILE_BASE64	Tout le contenu de profile.txt
APP_STORE_CONNECT_API_KEY_ID	L'ID de la clé (B3)
APP_STORE_CONNECT_API_ISSUER_ID	L'ID d'émetteur (B3)
APP_STORE_CONNECT_API_CONTENT	Tout le contenu de key.txt

D. Lancer la compilation

- 1. Dans le dépôt GitHub : onglet **Actions** ;
- 2. Dans la liste de gauche, cliquez sur **TestFlight** ;
- 3. Bouton **Run workflow** > branche main > **Run workflow** (vert) ;
- 4. Attendez environ 15 minutes (les machines GitHub compilent, signent et publient). **Ce que vous devez voir :** une coche verte à côté du run ; en cas de croix rouge, cliquez le run, ouvrez l'étape en

rouge et lisez la dernière ligne — c'est le message à me transmettre.

E. Installer sur l'iPhone

- 1. Sur appstoreconnect.apple.com : votre app **OSINT Suite** > onglet **TestFlight** : la version construite apparaît (« Prêt à être soumis » ou « En traitement » — patientez quelques minutes) ;
- 2. Section **Testeurs** : bouton + > ajoutez votre propre adresse e-mail ; le lien d'invitation part par mail ;
- 3. Sur l'iPhone : installez l'app **TestFlight** depuis l'App Store, ouvrez le mail, touchez le lien ;
- 4. **Installer** OSINT Suite ; répétez pour **Gestionnaire d'accès** (deux apps, un seul TestFlight) ;
- 5. Ouvrez le Gestionnaire d'accès, autorisez l'appareil, notez la phrase de récupération — puis OSINT Suite et « Demander le déverrouillage ».

Limites de l'option 2 : chaque mise à jour du code repart d'un push GitHub puis d'un build (~15 min). Et le rôle « Mac hub » (collecte des veilles) exige malgré tout l'app sur le Mac — que vous devrez construire une fois avec la méthode classique (ou accepter de n'utiliser que le rôle « consomme uniquement » sur iPhone).

F. Ça ne marche pas ? Pannes classiques de l'option 2

Message sur GitHub	Solution
Secret APPLE_TEAM_ID manquant	Le secret n'a pas été créé (étape C) : vérifiez Settings > Secrets > Actions.
« no signing certificate » / « no provisioning profile »	Le .p12 ou le profil ne correspond pas à votre Team : refaites B1/B2 en vérifiant que le certificat est bien Apple Distribution et le profil App Store Connect.
« …is not registered on your account » (bundle ID)	L'App ID n'existe pas sur le portail : créez fr.osintsuite.app (B4) avant de relancer.
Coche verte mais rien sur TestFlight	Attendez : le traitement Apple prend 5 à 30 minutes après le build ; vérifiez que l'app existe bien dans App Store Connect (B4).
« Invalid API key » à l'upload	Les trois valeurs B3 ont été mélangées : Key ID ≠ Issuer ID ≠ contenu du fichier .p8. Refaites l'étape C en vérifiant chaque secret une par une.

2.16 Tester sans compte développeur payant — le simulateur

Vous pouvez découvrir et utiliser l'application à 100 % sans compte Apple Developer payant, sur un iPhone et un iPad VIRTUELS (le simulateur, inclus dans Xcode). Lisez d'abord le tableau des trois situations pour choisir, puis suivez le pas-à-pas du simulateur.

A. Les trois situations, selon votre compte

Votre situation	Ce qui fonctionne	Ce qui ne fonctionne pas
Aucun compte Apple développeur	Simulateur iPhone/iPad : tout l'application, déverrouillage inclus	Aucun appareil réel
Compte Apple gratuit (ajouté dans Xcode)	Simulateur complet + lancement des apps sur votre Mac	iPhone réel : les apps s'installent mais le déverrouillage reste bloqué (voir B)
Compte payant (99 $/an)	Tout : Mac, iPhone, iPad, TestFlight, Mac hub	— (aucune limite)

B. Pourquoi l'iPhone réel est bloqué sans compte payant (à savoir)

L'architecture de sécurité repose sur le **partage de trousseau** (Keychain Sharing) entre l'OSINT Suite et le Gestionnaire d'accès : c'est ainsi que la clé de session transite entre les deux applications. Or Apple réserve cette capacité aux comptes payants — **aucun contournement légal n'existe**. Conséquence concrète avec un compte gratuit :

- Les deux applications se **compilent et s'installent** quand même sur un iPhone réel ;
- mais la clé de session **ne peut pas transiter** entre elles : l'OSINT Suite reste sur l'écran « Base verrouillée » ;
- ce n'est donc **pas la peine d'essayer** de compléter les étapes iPhone réelles sans compte payant — passez par le simulateur.

Sur le **simulateur**, aucun de ces problèmes : il n'y a pas de signature réelle, le trousseau partagé fonctionne, tout le flux (déverrouillage, veilles, carte, graphe) est testable — y compris Face ID (simulé).

C. Pas-à-pas du simulateur (gratuit, 15 minutes)

- **1.** Vérifiez que Xcode est installé (étape 2.2) — c'est le seul prérequis ; aucun compte développeur n'est nécessaire ;
- **2.** Ouvrez le Terminal et placez-vous dans le dossier du projet (étape 2.7 : « cd » + glisser-déposer le dossier V2) ;
- **3.** Générez le projet si ce n'est pas déjà fait :

```
xcodegen generate
```

- **4.** Ouvrez le simulateur iPhone :

```
open -a Simulator
```

Ce que vous devez voir : une fenêtre affichant un iPhone avec son écran d'accueil.

- **5.** Compilez et lancez le Gestionnaire d'accès dans le simulateur :

```
xcodebuild -scheme AccessManager-iOS \ -destination 'platform=iOS
Simulator,name=iPhone 16' build xcrun simctl launch booted fr.osintsuite.acces
```

- **6.** Puis l'OSINT Suite :

```
xcodebuild -scheme OSINTSuite-iOS \ -destination 'platform=iOS Simulator,name=iPhone
16' build xcrun simctl launch booted fr.osintsuite.app
```

- **7.** Dans le simulateur : ouvrez le Gestionnaire d'accès, autorisez l'appareil, générez la phrase de récupération ; puis OSINT Suite > « Demander le déverrouillage ».

Ce que vous devez voir : la base se déverrouille dans le simulateur et la page d'accueil (Acquérir / Capitaliser / Exploiter / Gérer) s'affiche. Vous pouvez tout essayer.

Astuce : pour changer de modèle d'iPhone, listez les simulateurs disponibles avec « xcrun simctl list devices available » et remplacez « iPhone 16 » par le nom voulu (iPad inclus — testez aussi la vue Split View).

D. Lancer l'OSINT Suite sur votre Mac (compte gratuit)

Sur le Mac, le trousseau partagé fonctionne avec un compte gratuit signé dans Xcode (Settings > Accounts > + > Apple ID). Suivez les étapes 2.6 à 2.9 du guide, en choisissant votre Team gratuit au moment de la signature.

- Les applications se lancent et fonctionnent pleinement sur le Mac ;
- Limite : le **Mac hub** (collecte périodique via LaunchAgent) et le Secure Enclave exigent une signature payante pour un usage de production — pour tester, tout est utilisable ;
- Rappel : sur Mac, contrairement à l'iPhone réel, le déverrouillage **fonctionne** avec un compte gratuit.

E. Récapitulatif — quelle voie choisir ?

Objectif	Voie	Coût
Découvrir / tester l'app	Simulateur (section C)	Gratuit — Xcode seul
Utiliser sur votre Mac	Étapes 2.6–2.9 + compte gratuit	Gratuit
Utiliser sur iPhone/iPad réels	Compte payant obligatoire (Keychain Sharing)	99 $/an
Installer sans Xcode (CI/TestFlight)	Option 2 (§2.14) — payant aussi	99 $/an

3. Configuration

3.1 Premier lancement du Gestionnaire d'accès

Au premier lancement, le Gestionnaire d'accès :

- génère la **KEK** (clé maîtresse) dans le **Secure Enclave** — non extractible, liée au matériel (repli KEK logicielle sur simulateur) ;
- génère le **secret maître MLA** (256 bits, base64) qui chiffre les archives, et l'enveloppe avec la KEK ;
- propose d'autoriser cet appareil.

Générez immédiatement la **phrase de récupération** (menu Actions) : 12 mots affichés **une seule fois**. Notez-les sur papier et conservez-les hors de tout appareil — ils sont le seul secours si le Secure Enclave devient inaccessible (changement d'appareil).

3.2 Déverrouillage de l'OSINT Suite

Le flux nominal à chaque session :

- L'OSINT Suite affiche l'écran « Base verrouillée » ;
- « Demander le déverrouillage » appelle l'App Intent du Gestionnaire d'accès ;
- Vous vous authentifiez (Face ID / Touch ID ou code) ;
- Une **clé de session à durée limitée** (1 h par défaut, réglable de 5 min à 24 h) est publiée dans le groupe de trousseau partagé ;
- L'OSINT Suite déchiffre les archives MLA **en mémoire** (jeu de travail et index reconstruits) ;
- Au passage en arrière-plan : purge de la session et **consolidation** des archives MLA avant fermeture.

La KEK ne quitte jamais le Gestionnaire d'accès : une application principale compromise ne peut pas ouvrir la base sans validation humaine côté compagnon.

3.3 Proxy Mistral (option A validée)

Déployez le proxy Vapor sur votre serveur (Fly.io, Render, VPS ~4 €/mois ou Mac local). La clé API Mistral ne quitte jamais ce serveur :

```
cd Server/ProxyVapor # Fichier .env (JAMAIS commité — .gitignore l'exclut) cat &gt;
.env &lt;&lt;'EOF' MISTRAL_API_KEY=votre_cle_mistral
TOKEN_CLIENT=token_aleatoire_genere_par_vous PORT=8080 EOF swift run # le proxy
écoute sur le port 8080
```

Testez la disponibilité :

```
curl https://votre-serveur.example.com/sante # réponse attendue : ok
```

Raccordement dans l'application : l'URL du proxy et le token client sont stockés dans le **Keychain** de l'app (jamais dans les réglages en clair). Deux façons de les y écrire :

- **Méthode simple (Paramètres de l'app) :** Gérer > Paramètres > Clés — saisissez l'URL du proxy et le token client ; ils sont écrits dans le Keychain (clés **url-proxy** et **token-proxy**) ;
- **Vérification (Terminal, sur Mac) :** après le premier lancement de l'app, vous pouvez contrôler la présence des entrées trousseau avec l'app « Trousseau d'accès » (chercher « com.osintsuite.secrets »).

Avec ce réglage, les fonctions suivantes deviennent actives :

- Résumés, NER et cotation assistée (Capitaliser) ;

• Traduction automatique des veilles multilingues ;

• **Mode découverte** (Acquérir > Découvrir) — sans proxy configuré, ce mode répond « indisponible » sans bloquer le reste ;

• Complétion automatique des fiches (la partie Wikipedia fonctionne même sans proxy, via Tor).

3.4 Collecteur « Mac hub » et LaunchAgent

iOS n'autorise pas l'exécution continue en arrière-plan : le Mac est le collecteur principal, les appareils mobiles consomment les résultats synchronisés (rafraîchissement opportuniste à l'ouverture + BGTaskScheduler).

```
xcodebuild -scheme Collecteur -configuration Release build cp
.build/release/collecteur /Applications/OSINTSuite.app/Contents/Resources/ #
Renseignez __CHEMIN_BASE__ et __PASSPHRASE__ dans la plist # (injectés par l'app
macOS au premier déverrouillage) cp App/LaunchAgent/com.osintsuite.collecteur.plist
~/Library/LaunchAgents/ launchctl load
~/Library/LaunchAgents/com.osintsuite.collecteur.plist
```

Le collecteur exécute toutes les veilles actives toutes les 15 minutes et insère les nouveaux éléments dans la base chiffrée locale.

3.5 Cartes et tuiles

Fond	Usage	Licence / attribution
OpenStreetMap	Plan (rues, toponymes), téléchargeable hors ligne	© OpenStreetMap contributors
IGN Orthophotos	Ortho-images France ~20 cm	Géoplateforme, licence Etalab 2.0
EOX Sentinel-2	Satellite mondial sans nuages ~10 m	CC BY 4.0, attribution obligatoire

Ajoutez le package MapLibre dans Xcode (https://github.com/maplibre/maplibre-native-ios) : les styles et le pont de cache tuiles sont déjà implémentés (ConfigMapLibre, PontTuilesMapLibre — boucle locale 127.0.0.1, cache d'abord, réseau ensuite). Utilisez « Précharger la zone » dans la vue Carte pour télécharger à l'avance les tuiles d'une zone et la consulter hors connexion.

3.6 Tor — trafic sortant obligatoire

Le client **Arti** (implémentation officielle Tor) est embarqué dans l'application et expose un proxy SOCKS5 sur 127.0.0.1 :

• **Tout** le trafic sortant (veilles, tuiles, proxy Mistral, GitHub, Wikipedia) transite par Tor — aucune exception ;

• Résolution DNS via Tor : pas de fuite DNS ;

• Si une plateforme bloque les sorties Tor : échec **journalisé** dans le journal d'exécution, aucune donnée perdue ;

• Ponts obfs4 configurables dans Paramètres (section Tor) ;

• **Pays de sortie** : réglage global (Paramètres) ou par veille — sans pays configuré, le client **Arti** offre l'anonymat maximal ; avec un pays (ex. « fr »), le client **tor C** restreint la sortie (ExitNodes). ■■ Restreindre le pays de sortie réduit l'anonymat : option explicite, désactivée par défaut ;

• Conséquences assumées : latence accrue (cache local systématique), APNs refusés par défaut (tirage périodique + notifications locales), CloudKit exclu (synchronisation via GitHub/Tor).

Compilation des xcframeworks Rust (MLA + Arti) :

```
rustup target add aarch64-apple-darwin x86_64-apple-darwin \ aarch64-apple-ios
aarch64-apple-ios-sim ./scripts/build-xcframeworks.sh # puis lier
Frameworks/mla.xcframework et Frameworks/tor.xcframework dans Xcode
```

3.7 Mode découverte — explorer une thématique inconnue

Le mode découverte (onglet Découvrir du module Acquérir) lance une recherche approfondie sur une thématique que vous tapez librement :

- L'application appelle Mistral en **mode réflexion** (raisonnement itératif) et effectue des **recherches internet via Tor** ;
- Une **synthèse structurée** est rédigée : les **sources sont citées** avec une cotation A–F **proposée** (vous validez, modifiez ou refusez — décision humaine toujours) ;
- À l'issue, des **sources d'intérêt** à surveiller sont proposées (sites, flux, comptes sociaux) : **créez les veilles en un clic** (formulaire pré-rempli) ;
- La synthèse est **enregistrée comme un document** : importable et capitalisable comme tout document, et vous pouvez créer des entités depuis son contenu (clic droit sur un terme).

Coût Mistral : une découverte consomme 5 à 15 appels API (boucle de raisonnement). Une estimation vous est systématiquement affichée AVANT le lancement, avec bouton de confirmation.

3.8 Nouvelles fonctionnalités de la base de connaissance

Individus — identité réelle, fictive ou numérique :

- La fiche Individu a un **type d'identité** : réelle, fictive ou numérique (compte social, pseudonyme en ligne) ;
- Un individu **fictif ou numérique est automatiquement proposé d'être lié à l'individu réel** correspondant (détection par similarité de nom) — vous validez ou corrigez le lien ; le lien apparaît dans le graphe relationnel.

Lieux — réel, fictif ou numérique :

- La fiche Lieu a un **type de lieu** ; un lieu **numérique** accepte une **adresse IP** à la place des coordonnées GPS (format vérifié automatiquement) ;
- La géolocalisation d'une IP reste un **choix explicite** de votre part — jamais automatique.

Sources liées automatiques :

- À la complétion d'une fiche, la page **Wikipédia** trouvée est ajoutée comme **source liée** (URL, langue, date de consultation, cotation proposée) ;
- Tout **compte social découvert** crée un individu **numérique** associé, lié à l'entité et visible dans le graphe.

Veille VK (■■■■■■■■■) :

- Connecteur via l'API officielle **wall.get** avec un token d'accès personnel créé dans vos réglages VK ;
- Surveillance d'un compte ou d'une communauté (identifiant dans la configuration de la veille) ;
- Risques documentés : CGU VK, token à durée de vie limitée — renouvelez-le périodiquement ; requêtes via Tor comme toujours.

Couche événements ACLED sur la carte :

- Affichage des événements du jeu de données **ACLED** (conflits et événements violents géolocalisés) ;

• Accès par l'API ACLED — **gratuit après inscription myACLED** (acleddata.com) ; la clé d'accès est conservée dans le **Gestionnaire d'accès**, jamais dans l'app ni le dépôt ;

• Téléchargement via **Tor**, stockage local chiffré (archive MLA), **mise à jour à la demande** ;

• Filtrable par **date, type d'événement et zone** (la zone affichée sur la carte) ;

• **Clic droit sur un événement** : création des entités Événement / Lieu correspondantes dans la base de connaissance ;

• Attribution obligatoire « ACLED » affichée en permanence sur la couche et dans tous les exports (politique ACLED).

3.9 Vos clés et comptes externes — récapitulatif du premier usage

Fonction	Ce qu'il faut fournir	Où c'est conservé
Actions Mistral (résumé, NER, traduction, découverte)	URL du proxy + token client (§3.3)	Keychain de l'app (url-proxy / token-proxy)
VK (■■■■■■■■■)	Token d'accès personnel créé dans vos réglages VK	Keychain de l'app
Discord	Token du bot + identifiant du canal	Keychain de l'app
Couche ACLED	Clé myACLED + e-mail du compte (inscription gratuite sur acleddata.com)	Gestionnaire d'accès
Secret maître MLA	Rien à fournir — généré au premier lancement	Gestionnaire d'accès (Secure Enclave)
Phrase de récupération	Rien — 12 mots affichés une seule fois	Sur papier, hors de tout appareil

4. Utilisation quotidienne

La page d'accueil présente quatre entrées en grille adaptative :

4.1 Acquérir — importer et collecter

Import de fichiers (PDF, images, TXT/Markdown, HTML, audio/vidéo) :

- Bouton « Choisir un fichier » ou glisser-déposer ;
- Champs obligatoires : **Nom**, **Date**, **Source** (libellé, réutilisée ou créée automatiquement), **Cotation source A–F** ;
- Le hash SHA-256 détecte les doublons d'import (fichier identique refusé) ; le texte est extrait automatiquement (TXT/MD/HTML).

Veilles — onglet Veilles :

- Créez une veille d'actualités (mots-clés), RSS/Atom (URL du flux) ou réseau social (X/nitter, Telegram t.me/s, Discord bot, TikTok import manuel) ;
- Cochez les **moteurs d'actualités** à interroger : Google, Qwant, Yandex (tous via Tor ; par défaut Google seul) ;
- Choisissez les **langues de recherche** (ex. « fr, en, de ») : la veille interroge chaque moteur dans chaque langue ;
- Option **pays de sortie Tor** (ex. « fr ») : force la sortie du trafic par un nœud du pays — utile quand un moteur ou une source bloque les visiteurs étrangers. ■■ Restreindre le pays de sortie RÉDUIT votre anonymat : l'option est désactivée par défaut ;
- Réglez la fréquence (5 min à 24 h) et les **mots-clés de priorité** ;
- « Exécuter toutes maintenant » pour un passage immédiat ; activez/désactivez individuellement ; consultez le **journal d'exécution** (date, succès, nombre de nouveaux éléments) ;
- Chaque nouvel élément atteignant le seuil de priorité déclenche une **notification locale** (3 max par passe, pour éviter le spam).

Veille multilingue — tout s'affiche en français :

- Les résultats trouvés dans une autre langue sont **traduits automatiquement en français** (via Mistral, à travers Tor) ;
- Le **texte original est conservé** avec sa langue d'origine : consultable depuis la fiche de l'élément ou le document source ;
- Hors ligne : les éléments déjà traduits restent lisibles ; les autres attendent le retour de la connexion (file d'attente).

Moteurs d'actualités — conditions et risques :

Moteur	Accès	Points d'attention
Google Actualités	Flux RSS officiel	Paramètres langue/pays par requête — le plus stable
Qwant Actualités	API publique v3 (JSON)	API non contractuelle : peut évoluer sans préavis ; échec journalisé sans interrompre les autres moteurs
Yandex Actualités	Flux RSS (paramètre région)	Disponibilité variable selon le pays de sortie — c'est le cas d'usage du pays de sortie par veille

Plateformes sans API publique : TikTok fonctionne par import manuel (collez « url \| texte », une publication par ligne) ; X passe par des flux RSS tiers (nitter) — instables ; Discord exige un bot (token dans le Keychain).

4.2 Capitaliser — structurer la veille

File d'attente : liste chronologique des éléments non traités.

- Sélectionnez les informations relatives à un même événement ;
- Demandez une **proposition de cotation Mistral** (résumé + cotation justifiée) — la décision finale reste humaine : validez, modifiez ou refusez dans le sélecteur A1…F6 ;
- « Capitaliser » crée le regroupement d'événement avec toutes ses sources et leurs cotations.

Base de connaissance — six types de fiches :

Type	Champs
Individu	Prénom, nom, biographie, commentaires
Organisation	Dénomination, type, synthèse, commentaires
Événement	Dénomination, date et heure, résumé, commentaires
Lieu	Dénomination, adresse ou coordonnées GPS, résumé, commentaires
Objet	Dénomination, type, précisions (immat., n° série), commentaires
Source	Dénomination, description, cotation A–F

- À la création d'une fiche, l'application **complète automatiquement** les champs (résumé Wikipedia FR puis synthèse Mistral) et propose la fiche pré-remplie à validation ; les champs auto-remplis portent le marqueur « [source : internet] » ou « [source : mistral] » jusqu'à validation ;
- La **reconnaissance d'entités** relie automatiquement les entités existantantes dont le nom apparaît dans le texte ; tout terme peut aussi être créé comme nouvelle entité (choix du type) ;
- Chaque fiche est **modifiable à la main**, avec historique des modifications (date, champs modifiés) ;
- Bouton « Exploiter » : ouvre directement le graphe, la frise ou la carte centrés sur l'entité.

4.3 Exploiter — analyser et visualiser

Graphe relationnel :

- Centré sur l'entité ciblée (rang 0), cercles concentriques par rang ;
- Ajoutez ou retirez des **rangs** (1–4) : rang 1 = directement liées, rang 2 = liées des liées, etc. ;
- Clic droit sur un nœud : **étendre le graphe** depuis cette entité ;
- Filtres par type d'entité et cotation minimale ; zoom par pincement ;
- Clic sur un nœud pour ouvrir la fiche.

Frise chronologique : événements et regroupements capitalisés, filtrables par type et cotation ; cartes cliquables vers la fiche.

Carte :

- Bascule de fond : OSM plan, IGN ortho (France ~20 cm), EOX satellite (monde ~10 m) ;
- Épingles cliquables des lieux géolocalisés → fiche ;
- « Précharger la zone affichée » : téléchargement à l'avance des tuiles pour consultation hors ligne ; attributions de licence affichées en permanence.

Limite assumée : en gratuit, la résolution satellite mondiale est de ~10 m ; la haute résolution (~20 cm) est réservée à la France (IGN).

4.4 Gérer — administrer

Doublons — validation explicite :

- Détection automatique à l'import et à la capitalisation : hash exact, similarité de texte (Jaccard), dénominations proches ;
- Écran de revue : « C'est un doublon » / « Ce n'est pas un doublon » — les paires refusées sont **mémorisées et jamais reproposées** ;
- Fusion assistée **champ par champ** : pour chaque champ, garder A ou B (concaténation possible pour les commentaires), inversion des rôles possible ; les relations et sources sont transférées, la fusion est journalisée.

Paramètres :

- Seuils de similarité doublons (texte, dénominations) ;
- Fréquence de veille par défaut ; actions automatiques Mistral activables/désactivables ;
- Synchronisation via dépôt GitHub privé (via Tor) activable — deltas CRDT Automerge chiffrés, jamais « dernier écrit gagnant » ;
- Rôle de l'appareil : **collecte** (Mac hub, veilles périodiques) ou **consomme uniquement** (iPhone/iPad, rafraîchissement à l'ouverture) ;
- Pays de sortie Tor global et par veille (avertissement de réduction d'anonymat affiché) ; moteurs d'actualités et langues par veille ;
- Export de la base (sauvegarde chiffrée) ; **effacement complet** (panic wipe) avec confirmation.

Le panic wipe est également disponible côté Gestionnaire d'accès : la révocation de la KEK rend la base définitivement illisible sur TOUS les appareils.

4.5 Utilisation sur iPhone/iPad au quotidien

Ouverture type sur mobile :

- Ouvrez OSINT Suite → « Base verrouillée » → « Demander le déverrouillage » ;

• Bascule vers le Gestionnaire d'accès → Face ID / code ; retour automatique — la base est ouverte pour la durée de session ;

• Le rafraîchissement de veille se déclenche à l'ouverture (via Tor) : les nouveaux éléments du Mac arrivent par synchronisation ;

• En déplacement : mode hors ligne complet — fiches, graphe, frise et carte consultables depuis les caches locaux ; saisissez librement, la synchronisation repartira à la reconnexion ;

• Les alertes de mots-clés de priorité arrivent en notifications locales lors des rafraîchissements.

4.6 Gestionnaire d'accès — coffret de clés

Fonction	Description
Appareils autorisés	Liste des appareils habilités, révocation individuelle
Profils	Lecture seule ou complet, plage horaire éventuelle (à cheval sur minuit supportée)
Rotation des clés	KEK (l'ancienne devient inutilisable) et DEK, à la demande
Phrase de récupération	12 mots, affichés une seule fois, à conserver hors appareil
Journal d'audit	Chaque déverrouillage (date, appareil, succès/échec), exportable JSON
Panic wipe	KEK révoquée = base définitivement illisible partout

Limites documentées :

• Le groupe de trousseau partagé est accessible à toutes les apps du groupe : seule la clé de session (transitoire) y est placée, jamais la KEK ;

• Les éléments du trousseau persistent après désinstallation d'une application (limite connue d'iOS) — cf. Aide utilisateur ;

• Friction d'usage : chaque déverrouillage passe par le Gestionnaire d'accès ; compensée par la durée de session réglable (5 min à 24 h).

5. Dépannage et bonnes pratiques

Symptôme	Cause probable	Solution
« Session absente ou expirée »	Session purgée (arrière-plan) ou expirée	Redemander le déverrouillage via le Gestionnaire d'accès
« Secret MLA non initialisé »	Premier lancement sans passage par le compagnon	Ouvrir d'abord le Gestionnaire d'accès (il initialise KEK/secret MLA)
« Échec via Tor » en boucle	Plateforme bloquante ou circuit coupé	Vérifier les ponts obfs4 (Paramètres > Tor) ; consulter le journal d'exécution
Un moteur d'actualités ne renvoie rien	Blocage géographique (source refuse les visiteurs étrangers)	Configurer le pays de sortie Tor sur la veille (ex. « fr ») — en acceptant la réduction d'anonymat
Résultats dans une langue inconnue	Veille multilingue : original conservé, traduction en attente	L'affichage est en français après traduction Mistral ; l'original reste consultable sur la fiche de l'élément
Base illisible après changement d'appareil	Secure Enclave lié à l'ancien matériel	Restaurer depuis la phrase de récupération (12 mots)
« Cette plateforme n'expose pas d'API publique »	TikTok sans API	Utiliser l'import manuel (url | texte)
Veilles sans nouveaux éléments	Contenu déjà vu (hash) ou veille inactive	Vérifier l'activation et le journal d'exécution
Échec des actions Mistral	Proxy indisponible ou token invalide	curl https://…/sante ; vérifier .env et le Keychain
Tuiles manquantes hors ligne	Zone non préchargée	« Précharger la zone affichée » avant de partir hors connexion
Doublon reproposé après refus	Paires refusées mémorisées par paire non ordonnée	Comportement normal : vérifier que la décision a bien été enregistrée

Bonnes pratiques de sécurité :

- Ne jamais committer le fichier .env ni aucune clé (le .gitignore les exclut — vérifiez avant chaque push) ;
- Conservée hors appareil, la phrase de récupération est le seul recours : ne pas la photographier ni la stocker en ligne ;
- Révoquez immédiatement un appareil perdu depuis le Gestionnaire d'accès ;
- Exportez régulièrement la base (Paramètres) et conservez la sauvegarde chiffrée en lieu sûr ;
- Testez le panic wipe de temps en temps sur un environnement de test pour vérifier votre procédure de restauration.

Pour toute erreur de compilation après génération du projet, reportez-vous aux messages du compilateur et à la CI GitHub Actions (9 cibles de test + build du Collecteur), puis ouvrez un ticket sur le dépôt avec le

message complet.